第六讲教学设计：数据的桥梁——JDBC数据库连接技术 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: 数据的桥梁：JDBC数据库连接技术
- **授课时长**: 20分钟
- **适用对象**: 已掌握Java基础，需要学习数据持久化的Web开发初学者

一、教学目标

知识目标

1. 理解在Web应用中持久化数据（Data Persistence）到数据库的必要性。
2. 掌握JDBC（Java Database Connectivity）的概念、作用和核心架构。
3. 熟记使用JDBC连接数据库的六个核心步骤（Driver, Connection, Statement, ResultSet）。

能力目标

1. 能够独立编写Java代码，通过JDBC连接数据库并执行简单的查询操作。
2. 能够识别并处理常见的JDBC异常，并正确关闭数据库资源。

素质目标

1. 培养学生严谨的编程习惯，特别是资源管理的意识。
2. 建立"程序-API-驱动-数据库"的抽象思维模型。
3. **[思政融合]** 强调数据安全的重要性，任何技术的使用都必须遵守国家法律法规，保护用户信息安全，这是技术人员的基本职业道德和社会责任。

二、教学重点与难点

教学重点

- JDBC作为"桥梁"的核心角色。
- JDBC操作数据库的六个明确步骤。
- 在代码中正确管理数据库连接和资源释放(`try-catch-finally` 的应用)。

教学难点

- 理解JDBC的抽象架构，特别是 `Driver` 和 `DriverManager` 的作用。
- 将六个抽象步骤与具体的Java代码一一对应起来。

三、教学内容与时间安排

第一部分：问题引入 - 数据该去向何方？ (3分钟)

内容要点

1. **情景回顾 (Review):**
    - 提问学生："同学们，我们前面学习了JavaBean和会话管理技术。大家思考一下，无论是用户信息还是购物车数据，当服务器关闭或者浏览器会话结束后，这些数据会怎么样？"
    - 引导学生得出结论：数据会丢失。
2. **提出问题 (Problem):**
    - "那么，对于网站的核心数据，比如用户信息、商品列表、订单记录等，我们希望它们能被永久保存下来，而不是昙花一现。大家有什么解决方案吗？"
    - 引导学生想到：需要一个"仓库"来存储数据，这个仓库就是**数据库**。
3. **引出主题 (Topic):**
    - "我们知道了数据要存到数据库里。但新的问题又来了：我们的Java程序，它本身并不懂如何与MySQL、Oracle这些形形色色的数据库直接对话。怎么办？"
    - "今天，我们就来学习如何搭建一座连接Java程序和数据库的桥梁——**JDBC技术**。"

教学方法

- 问题导向法，从已有知识的局限性出发。
- 设问-解答式，引导学生自主思考，层层递进。

第二部分：JDBC核心揭秘 - 连接的艺术 (9分钟)

内容要点

1. **什么是JDBC?**
    - **定义**：JDBC (Java Database Connectivity) 是Java语言中用于规范客户端程序如何访问数据库的应用程序接口（API）。
    - **核心比喻**：它是一套**标准**和**规范**。就像USB接口一样，无论你用的是什么牌子的U盘或电脑，只要都遵守USB标准，就能互相通信。JDBC就是Java访问所有数据库的"USB标准"。
2. **JDBC架构**
    - 展示JDBC架构图，讲解四个核心组件：
        - **Java应用程序 (Application)**：就是我们编写的业务代码。
        - **JDBC API**：Java官方提供的一系列标准接口，如 `Connection`，`Statement`，`ResultSet` 等。
        - **驱动管理器 (Driver Manager)**：负责管理各种数据库的驱动程序。
        - **数据库驱动 (Driver)**：由数据库厂商提供的、实现了JDBC API的"翻译官"。
3. **JDBC编程六步曲 (重点)**
    - 这是本节课的重中之重，必须掌握。我们将详细拆解每一步。
    - **第一步：加载驱动**
        - **代码**: `Class.forName("com.mysql.cj.jdbc.Driver");`
        - **作用**: 请求JVM加载指定的驱动类。从JDBC 4.0开始，这一步是**可选的**，因为驱动可以通过SPI（Service Provider Interface）机制自动加载。但显式加载有助于理解原理。
    - **第二步：建立连接**
        - **代码**: `Connection conn = DriverManager.getConnection(url, user, password);`
        - **作用**: `DriverManager` 会扫描所有已加载的驱动，找到能处理指定URL的驱动，然后建立一个到数据库的物理连接。

        - **URL详解**: `jdbc:mysql://localhost:3306/mydatabase?useSSL=false&serverTimezone=UTC`
            - `jdbc:mysql://`：协议和子协议。
            - `localhost:3306`：主机名和端口。
            - `mydatabase`：数据库名称。
            - `?`：参数分隔符，后面跟键值对。
    - **第三步：创建执行对象**
        - **代码**: `Statement stmt = conn.createStatement();`
        - **作用**: 创建一个 `Statement` 对象，用于向数据库发送SQL语句。
    - **第四步：执行SQL语句**
        - **代码**: `ResultSet rs = stmt.executeQuery("SELECT ...");` 或 `int count = stmt.executeUpdate("INSERT/UPDATE/DELETE ...");`
        - **作用**: `executeQuery` 用于查询，返回结果集；`executeUpdate` 用于增删改，返回受影响的行数。
    - **第五步：处理结果集**
        - **代码**: `while(rs.next()) { rs.getString("column_name"); ... }`
        - **作用**: `ResultSet` 对象维护一个指向结果数据行的游标，`next()` 方法将游标移动到下一行。如果存在下一行，则返回 `true`。
    - **第六步：关闭资源**
        - **代码**: `rs.close(); stmt.close(); conn.close();`
        - **原则**: **后开先关**，并且必须在 `finally` 块中执行以确保资源一定被释放。

教学方法

- 比喻教学法 (USB标准、翻译官)。
- 结构化讲解 (六步曲)，条理清晰。
- 图示法 (展示JDBC架构图)，化抽象为具体。

第三部分：动手实践 - 编写第一个JDBC程序 (5分钟)

第三部分：JDBC代码实战与重构 (8分钟)

内容要点

1. **基础实现**：展示一个包含完整六个步骤的JDBC查询代码。

    ```
    // ... try-catch-finally and 6 steps ...
    ```

2. **升级一：使用 `PreparedStatement` 防止SQL注入**
    - **引入问题**: 展示使用 `Statement` 进行字符串拼接的危险代码：`String sql = "SELECT * FROM users WHERE username = '" + userInput + "'";` 讲解如果 `userInput` 是 `'' or '1'='1'"`，就会导致SQL注入漏洞。
    - **解决方案**: `PreparedStatement`
        - **原理**: 它会对SQL语句进行预编译，并将用户输入作为参数传递，从根本上杜绝SQL注入。同时，多次执行同一结构的SQL时性能更好。
        - **代码对比**:

```java
// PreparedStatement安全写法
String sql = "SELECT * FROM users WHERE username = ?"; // 使用?做占位符
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, userInput); // 设置参数
ResultSet rs = pstmt.executeQuery();
```

3. **升级二：封装JDBC工具类（`DBUtil`）**
   - **引入动机**: JDBC的资源关闭代码既冗长又重复。在实际工程中，我们会将其封装成工具类。
   - **代码示例** `DBUtil.java`：

```java
public class DBUtil {
    // ... 静态加载驱动 ...
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PWD);
    }

    public static void close(Connection conn, Statement stmt, ResultSet rs)
{
        if (rs != null) { try { rs.close(); } catch (SQLException e) {
e.printStackTrace(); } }
        if (stmt != null) { try { stmt.close(); } catch (SQLException e) {
e.printStackTrace(); } }
        if (conn != null) { try { conn.close(); } catch (SQLException e) {
e.printStackTrace(); } }
    }
}
```

   - **重构后的业务代码**: 展示使用 `DBUtil` 后，主逻辑变得多么简洁清晰。

```java
Connection conn = null;
// ...
try {
    conn = DBUtil.getConnection();
    // ... 业务处理 ...
} catch (Exception e) {
    // ...
} finally {
    DBUtil.close(conn, pstmt, rs);
}
```

4. **强调资源关闭**：重点讲解 `try-catch-finally` 代码块的必要性，演示如何正确、安全地关闭 `ResultSet`，`Statement`，和 `Connection` 资源。

教学方法

- **现场编码/代码走查 (Live Coding/Code Walkthrough)**，让学生看到一个功能完整的最小实现。
- **代码重构演进法**: 从最基础的实现，一步步重构到更安全、更优雅的工业级写法，让学生体会代码优化的过程和价值。
- **理论联系实际**：将前面讲的六个步骤与真实代码一一对应，巩固知识。

第四部分：课堂总结与展望 (3分钟)

总结要点

1. **为何需要JDBC?** 实现数据持久化，连接程序与数据库。
2. **JDBC是什么?** 一套连接数据库的**标准API**，是程序与数据库之间的桥梁。
3. **如何使用JDBC?** 牢记并实践"**编程六步曲**"，并安全地关闭资源。
4. **如何用得更好?** 优先使用 `PreparedStatement` 替代 `Statement` ，并将重复代码封装到工具类中。

展望 (承上启下)

- "今天我们学习了JDBC的基本用法，但是大家有没有发现，即使封装了 `DBUtil` ，我们每次连接数据库都要重复地获取连接、创建Statement、关闭连接。而且每次数据库交互都会建立一个新的物理连接，性能开销很大。"
- "有没有更高效、更安全的数据库连接方式呢？下一讲，我们将学习数据库连接池技术（如Druid），它将帮助我们管理数据库连接，大幅提升应用性能和代码的健壮性。敬请期待！"

四、教学方法与手段

教学方法

1. **问题驱动法 (Problem-Driven)**: 从数据持久化需求入手
2. **讲授法 (Lecturing)**: 系统讲解JDBC六步曲
3. **案例驱动法 (Case-Driven)**: 完整数据库查询案例
4. **比喻法 (Analogy)**: "USB标准"比喻JDBC规范
5. **双语教学法 (Bilingual Teaching)**:
    - 课堂采用英文授课，关键技术术语提供中英文对照
    - 重要概念（Driver, Connection, Statement, PreparedStatement, ResultSet）首次出现时给出英文表达和中文释义
    - 鼓励学生使用英文术语描述JDBC操作流程
    - 代码注释和变量命名使用规范英文

教学手段

- 多媒体课件 (Marp Slides/PPT)
- IDE (IntelliJ IDEA/Eclipse)
- 数据库客户端 (Navicat/DBeaver)

五、教学评价

- **过程性评价**: 课堂提问的回答情况；能否理解JDBC作为"桥梁"的角色。
- **终结性评价**: 能否用自己的话复述JDBC编程的六个步骤；能否独立编写代码完成一次数据库查询。

六、课后拓展

思考题

1. 除了防止SQL注入和提升性能，`PreparedStatement` 相比 `Statement` 还有哪些好处？（提示：代码可读性、类型安全）
2. 了解一下JDBC中的事务处理。`Connection` 对象有哪些方法（如 `setAutoCommit`，`commit`，`rollback`）可以用来管理事务？它们在什么场景下至关重要？

实践作业

1. **重构作业**: 将上次使用 `Statement` 完成的查询作业，全部重构为使用 `PreparedStatement`。
2. **实现增删改**: 使用 `PreparedStatement` 和 `executeUpdate()` 方法，为你自建的表（如 `students` 表）实现完整的增加、删除、修改功能，并编写 `main` 方法进行测试。